Image Processing - Lecture 10

Amir Atapour-Abarghouei

amir.atapour-abarghouei@durham.ac.uk

Reading:

Szeliski - Sec 2.3.3 (basic only)
Gonzalez / Woods - Sec 8.1 - 8.2.1
Solomon / Breckon - Sec 1.3.2

Image Compression for Storage & Transmission

Slide material acknowledgements: Fisher; Breckon; Atapour;

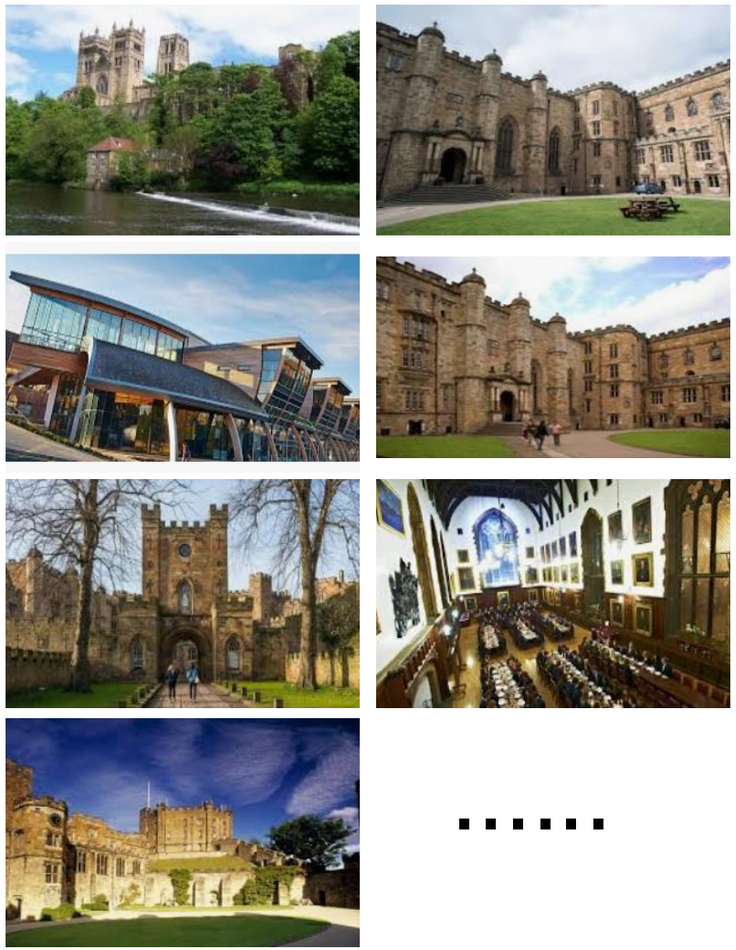
from this ...

file size = 2.3MB

… to this

file size = 500KB

Efficient Storage

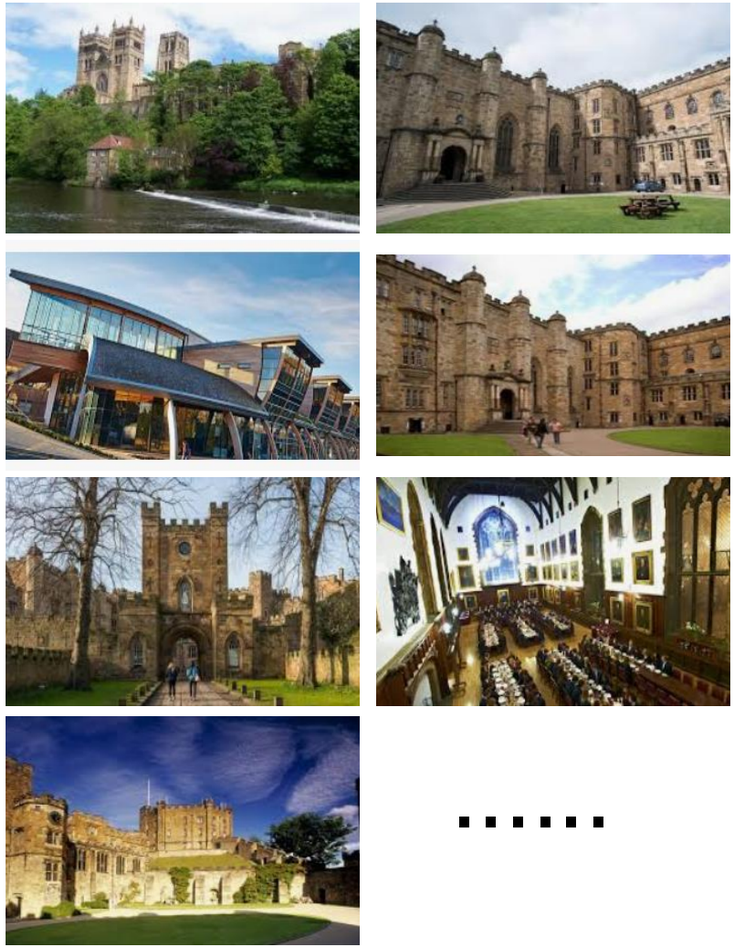


.....

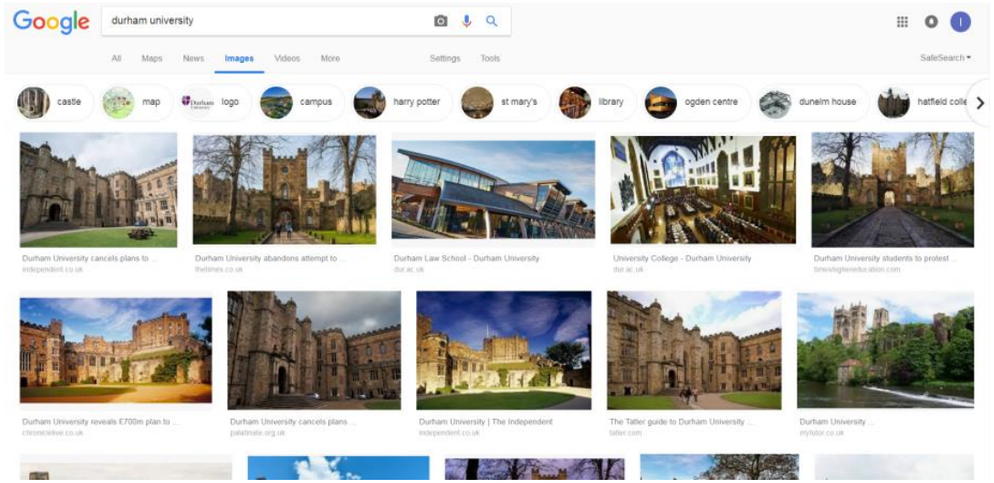
The goal is for more images taking up less storage space!

By 2030, the world will produce approximately 1 Yottabyte of data annually!

Faster Transmission



.....



Redundancy in Images

We can compress images by targeting three principal types of **redundancy**:

- **Coding Redundancy:** sub-optimal codes lead to using more bits than needed.

  efficient coding techniques → avoid representational redundancy

i.e. what is the minimum number of bits required to represent pixels?

- **Spatial Redundancy:** neighbouring pixels are likely to have similar values (spatial correlation). **Repeated information in spatially correlated pixels.**

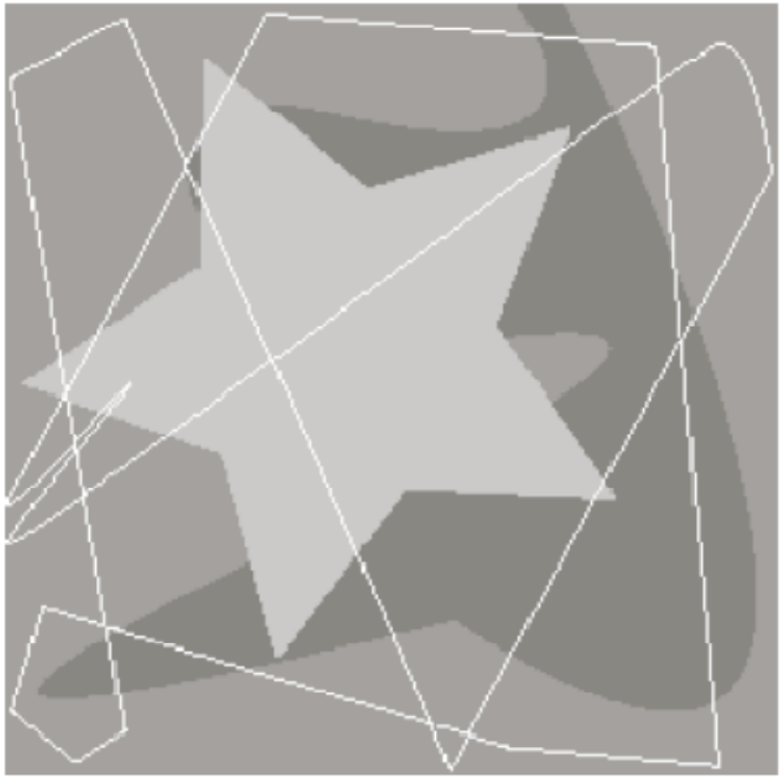
- **Irrelevant Information:** images may contain visually non-essential information that is ignored by the human visual system.

  Sacrifice finer image details → avoid visual redundancy

i.e. what is the minimum image information required? (application dependent!)

Example - Redundancy in Images

Coding redundancy



only 4 distinct intensities
2-bit instead of 8-bit image.

Spatial redundancy

single intensity value
across a whole row.

Irrelevant information

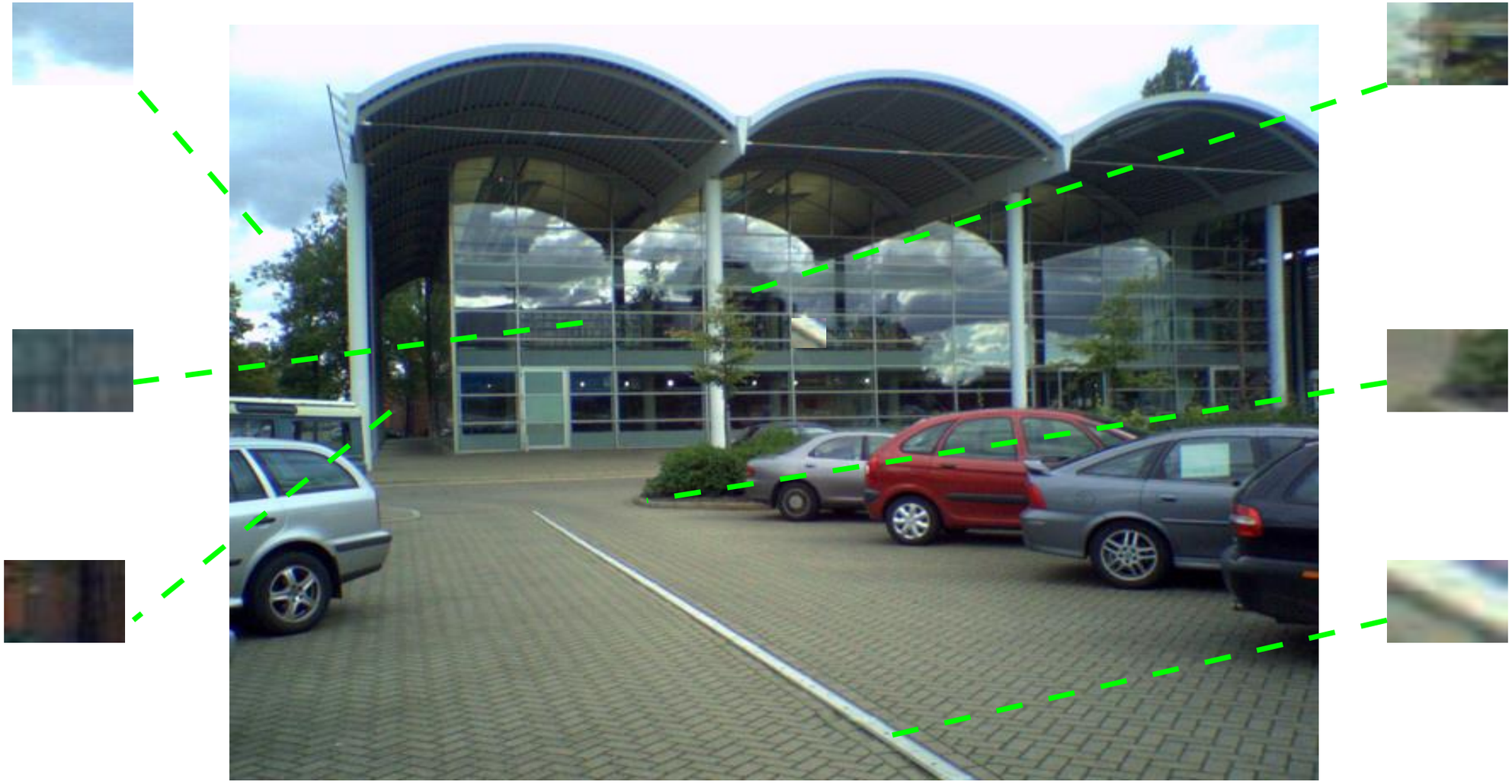
fine grained noise/texture
ignored by the eye.

Exploit Image Repetition

(via efficient pixel coding)

Exploit Image Redundancy

(via removal of image detail)



Lossy Compression

JPEG quality level : 0 - file size = 16KB

JPEG quality level : 20 - file size = 40KB

JPEG quality level : 60 - file size = 88KB

JPEG quality level : 75 - file size = 168KB

Lossy compression: sacrifice image quality for a smaller file size.

JPEG is an example of lossy compression.

Lossless Compression

PNG compression level : 0 - file size = 2.3MB

PNG compression level : 9 - file size = 1.1MB

Raw image size = 1024 x 768 (8 bit RGB) = 1024 x 768 x 3 = 2304KB ~= 2.3MB

Exact replication of data when uncompressed. No loss of information.

Lossy vs. **Lossless**

Lossy compression:

- Images need not be reproduced exactly - ***approximation*** *is OK.*
- compression **artefacts** in the image, which **may not be visible.**
- source of **noise** for image processing and computer vision algorithms.
- widely used with efficient implementations.

Lossless compression:

- computationally more **expensive.**
- resulting **file size larger** than lossy compression files.
- **no additional noise** to the image.
- less widely used but *very* important in some domains, e.g., medical images.

Modern Image Compression

JPEG compression

and

Discrete Cosine Transform (DCT)

JPEG

JPEG is an image compression algorithm based on the **Discrete Cosine Transform** and **variable length encoding**.

Offers tuneable (quality controlled by user) lossy compression.

JPEG is the most widely-used image compression technique.

The name JPEG is an acronym of the Joint Photographic Experts Group.

Discrete Cosine Transform (DCT)

- DCT expresses a vector of data points, here pixel intensity values, as a weighted sum of cosine functions of varying frequencies.
  - Similar concept to the DFT (Lectures 7 & 8).
  - DCT **uses cosine only** instead of sine and cosine functions (as per DFT).

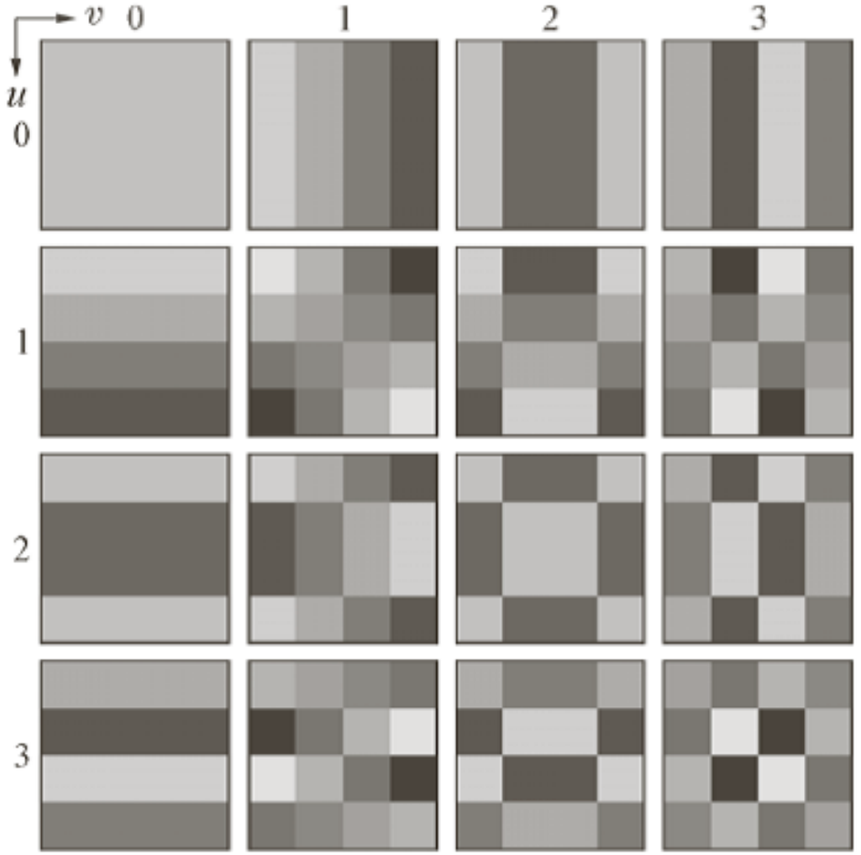
- Like DFT, DCT is a mathematical change of basis - multiplication of the data vector by a special matrix (DCT matrix).
  - Unlike the DFT, DCT is a real transform - real numbers elements.

- Why only cosine functions? **fewer cosine functions** are needed to approximate a typical signal or image – which helps with compression.

- DCT() and DCT^{-1}() exist as transform functions (details beyond our scope).

Basis Functions

- In 2D, DCT uses a set of 2D matrices as basis functions, each one corresponding to a 2D cosine function.

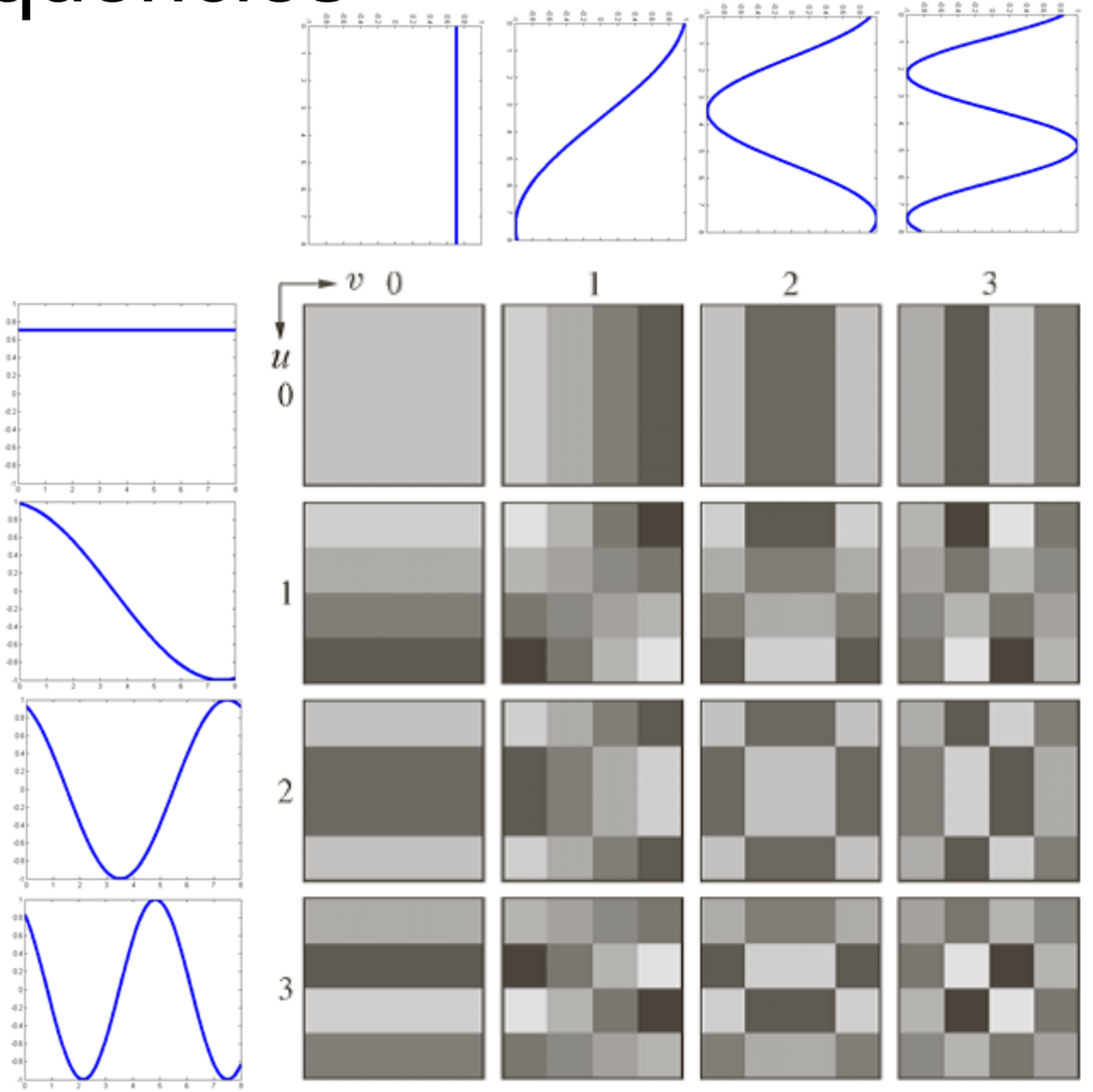


  - The basis of the 4×4 DCT, visualised as images.
  - It consists of 16 matrices, each one of dimension 4×4.
  - DCT: get the 4×4 image as a (weighted) combination of these 16 images?

Cosine Frequencies

The underlying cosine functions of each matrix row and column.

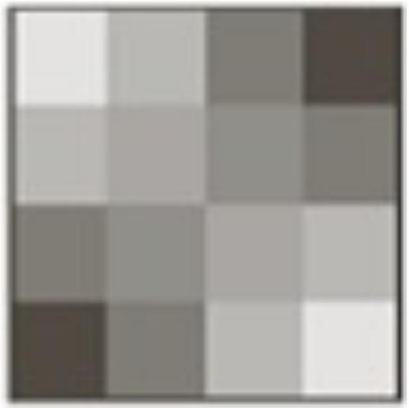
The functions are shown with increasing frequency.



Example Matrix

For example, the basis of the 4×4 DCT consists of 16 matrices (-ve / +ve). One such matrix is shown below.

0.250	0.135	-0.250	-0.326
0.135	0.073	-0.135	-0.176
-0.250	-0.135	0.250	0.326
-0.326	-0.176	0.326	0.426



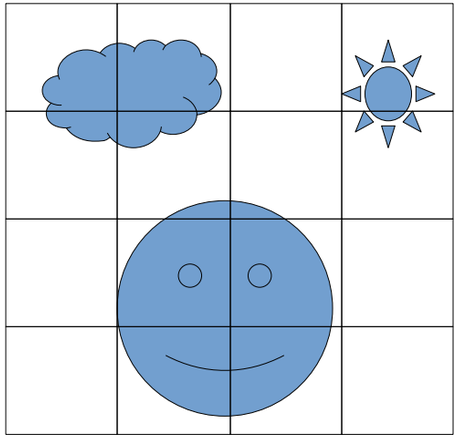
The (1,1) element of the basis of the 4x4 DCT

Here add -0.5 to the matrix to put it in the range [0,1] and multiply by 255 for display as greyscale.

As per our earlier DFT discussion (Lecture 7 / 8) an $n \times n$ image can now be approximated by a weighted sum of each of these matrices.

DCT Example

4 × 4 image



I_{input}

$$= \sum_{u=0}^{3} \sum_{v=0}^{3} x_{(u,v)} T(u,v)$$

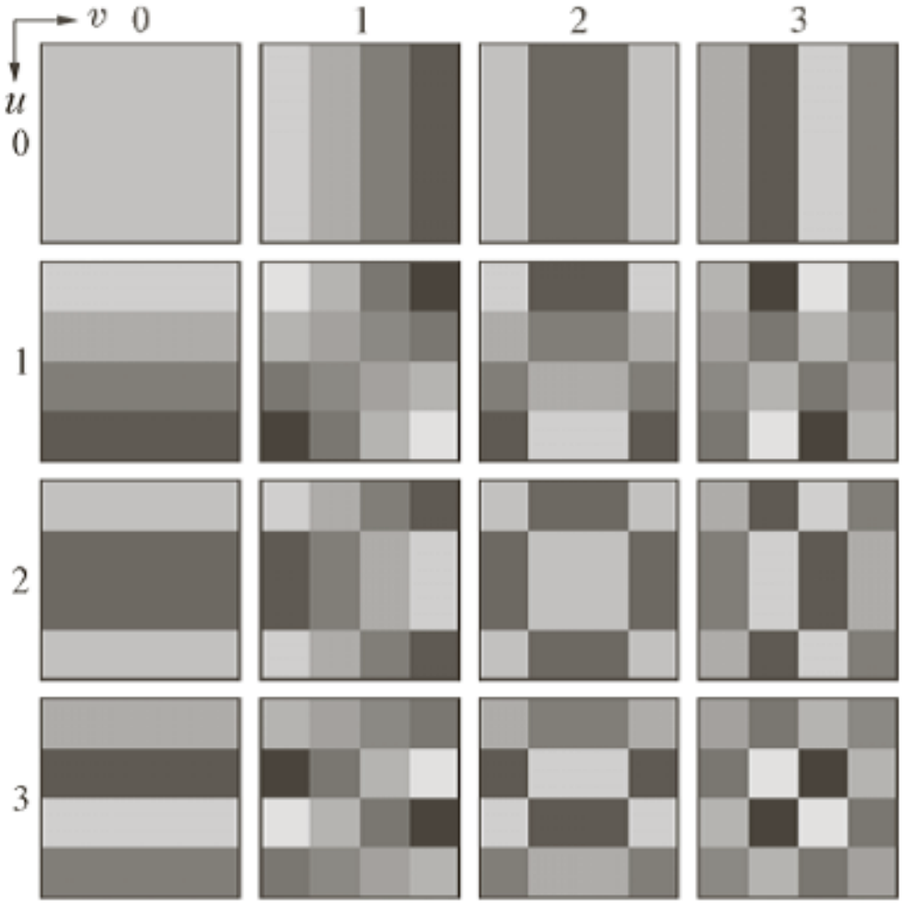
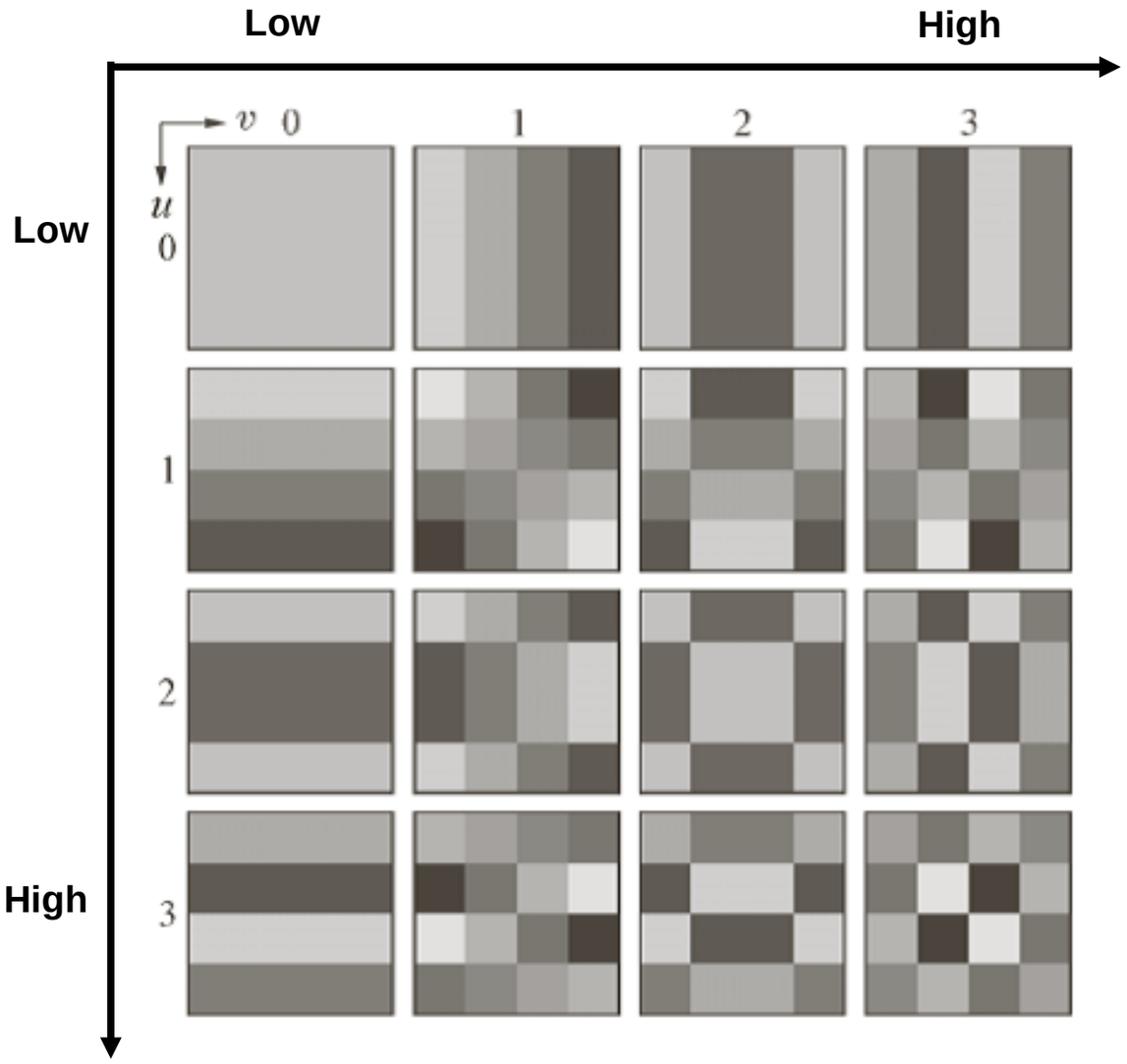


Image I_{input} can now be represented as a set of DCT coefficients $\{x_{(u,v)}\}$

- Each corresponding to the associated DCT basis matrix
- Same concept as with DFT coefficients (Lecture 7 / 8)

Low and High Frequencies

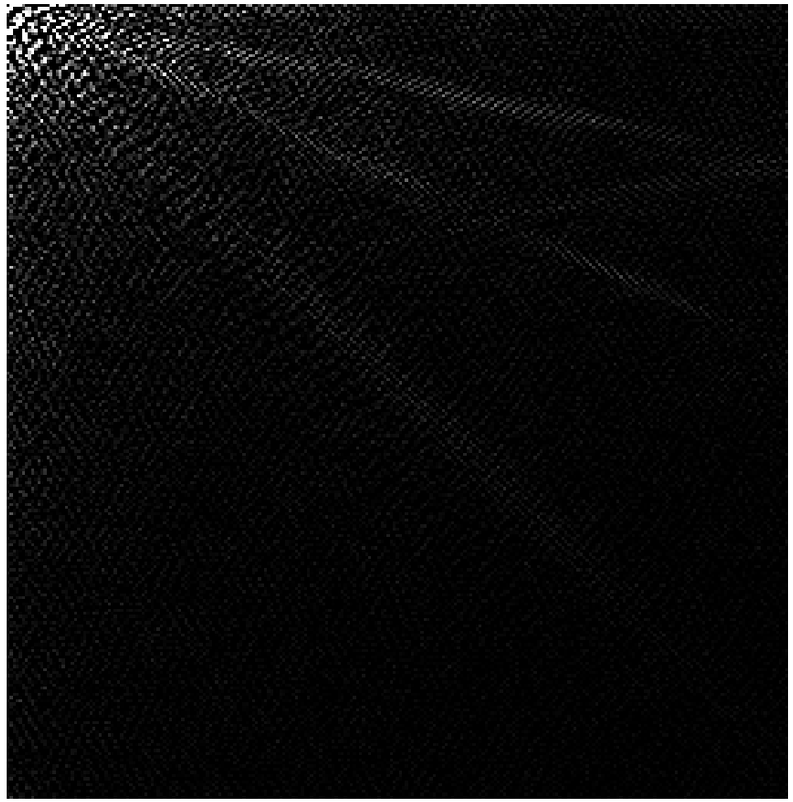


When a 1D (signal) DCT transform is written as a column vector, the low frequencies correspond to the top and the high frequencies to the bottom of the output.

Therefore, in the 2D (image) DCT the low frequencies correspond to the top-left of the image transform, while the high frequencies correspond to the bottom right.

Low and High Frequencies

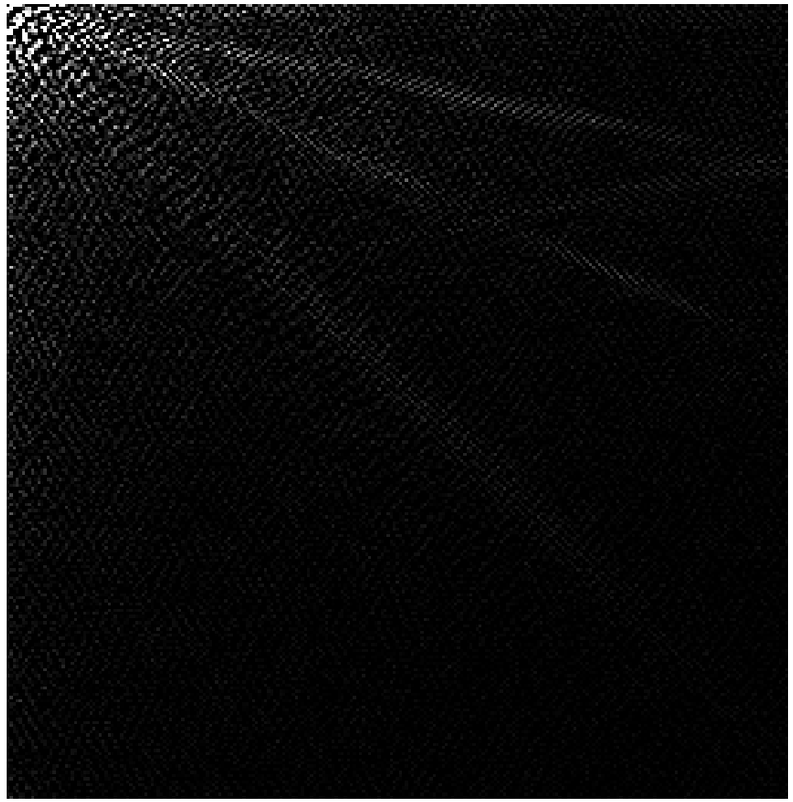
DCT



The larger coefficients, corresponding to lower frequencies, are concentrated on the upper left corner of the image of the DCT transform.

Low and High Frequencies

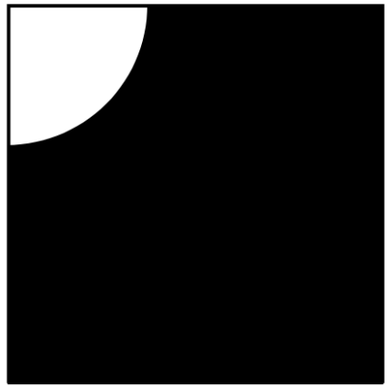
DCT



From the top-left corner of DCT output

High and low (spatial) frequencies represent the same portions of image information as per the DFT (lectures 7/8) - ***low pass and high pass filters!***

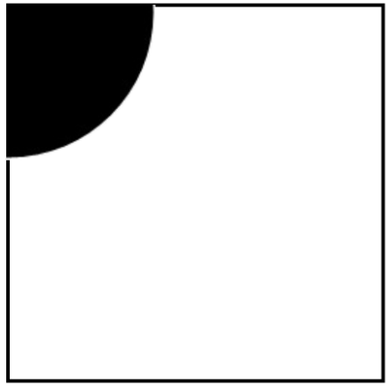
DCT Filtering - Same Principle as DFT



Ideal lowpass filter

Low Pass Filtering:

1. Compute the DCT of an image.
2. Multiply, the top left coefficients by one and all others by zero.
   [In other words, keep the low frequency coefficients only.]
3. Inverse the DCT.

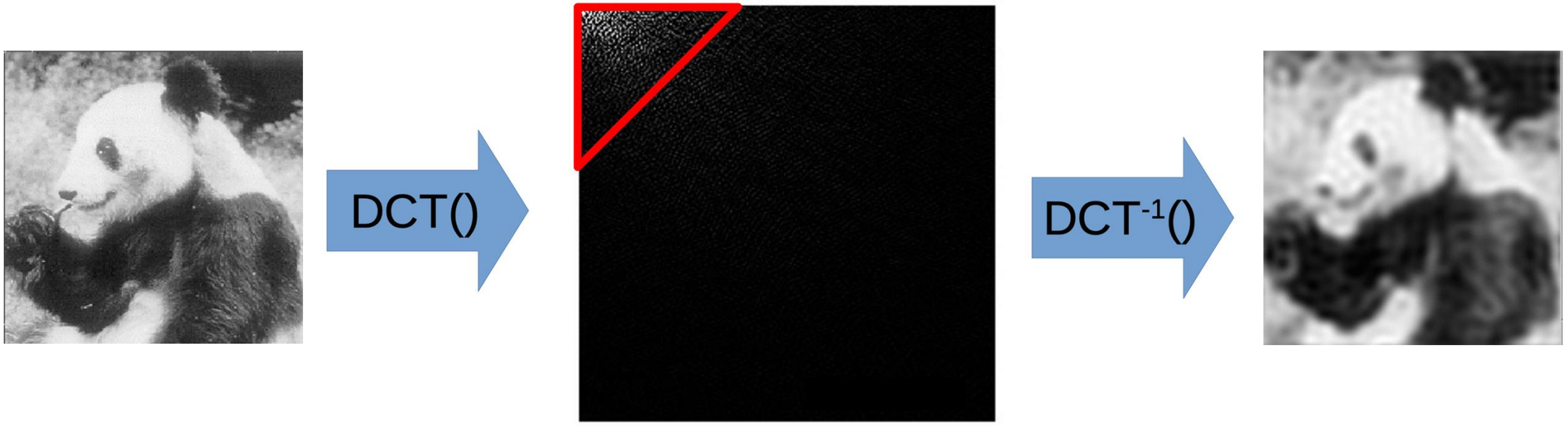


Ideal highpass filter

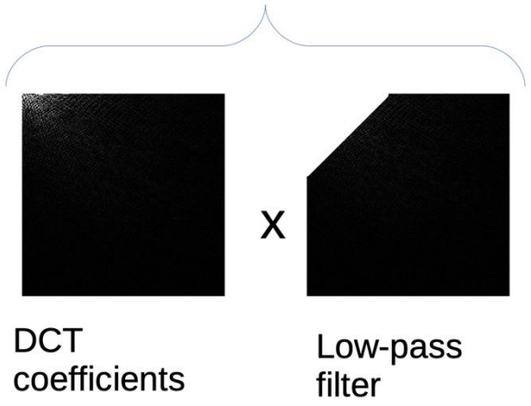
High pass filtering:

1. Compute the DCT of an image.
2. Multiply, the top left coefficients by zero and all others by one.
   [In other words, keep the high frequency coefficients only.]
3. Inverse the DCT.

Example: DCT Low pass Filtering

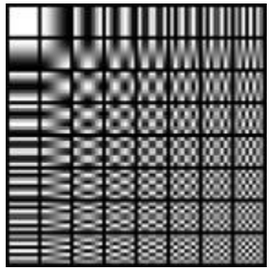


Only the DCT coefficients inside the red triangle are retained.



The high pass filter works in the opposite manner, of course!

JPEG Compression (8-bit Grayscale)



The basis functions of the 8x8 DCT

Step 0: Colour images are converted RGB $\rightarrow YC_RC_B$

- Illumination component (Y) + two colour components, (C_RC_B)
- Chroma subsampling: Colour reduced (quantised) by factor of 2 or 3

Step 1: Subdivide the image into **pixel blocks of 8×8** size. The blocks are processed one after the other from left to right and top to bottom.

Step 2: Assuming that the values of the pixels are in the range [0, 255], subtract 128 to bring them in the range [-128 , 127].

- DCT maps the interval [-128 , 127] – centred at zero.

Step 3: Apply the 8×8 DCT to each 8×8 block. The DCT values are computed with 11-bit precision (even though the input has 8-bit precision).

- Each image block is *approximated* by a subset of DCT basis matrices – **image redundancy will be exploited here** (loss of detail).

JPEG Compression (8-bit Grayscale)

Step 4: Scale and quantise the DCT values, using the following **scaling array:**

$$Z = \begin{bmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{bmatrix}$$

That is, if $T(u,v)$ is the DCT of the 8×8 pixel block, we do component-wise division of the two matrices and round the result:

$$\hat{T}(u,v) = \text{round}\left[\frac{T(u,v)}{Z(u,v)}\right]$$

JPEG Compression (8-bit Grayscale)

- As a result of Step 4, high-frequency coefficients are encoded with lower accuracy than the low-frequency components.
- Human vision is less sensitive to high frequency variations than low frequency ones.
- In JPEG, the overall image quality is controlled by a single user parameter:
  - The matrix Z shown in the previous slide corresponds to the quality parameter set to 50.
  - The matrix Z and its derivatives used for compressing at other levels of quality, were obtained empirically after extensive experimentation.

JPEG Compression (8-bit Grayscale)

Step 5: Create a sequence of the quantised DCT coefficients $\hat{T}(u,v)$, using the zig-zag pattern:

0	1	5	6	14	15	27	28
2	4	7	13	16	26	29	42
3	8	12	17	25	30	41	43
9	11	18	24	31	40	44	53
10	19	23	32	39	45	52	54
20	22	33	38	46	51	55	60
21	34	37	47	50	56	59	61
35	36	48	49	57	58	62	63

We expect long runs of zeros at the end of the sequence where the high frequencies are encoded.

JPEG compression (8-bit grayscale)

Step 6: Encode the sequence of the quantised coefficients $\hat{T}(u,v)$ obtained in Step 5 with a Huffman-based* variable length code, encoding each coefficient's value and the number of preceding zeros.

Use a special symbol for the end of non-zero coefficients.

The coefficients are specifically ordered (zig-zag) and *encoded* based on variable length predictive encoding – **image repetition is exploited here**

***Huffman Encoding**

analyse frequency of input symbols in stream, assign short codes to common ones and longer codes to less common ones: variable length encoding

- most frequent element → shortest code
- least frequent element → longest code

Example: Image Compression

An image, 1024 pixel × 1024 pixel × 24 bit:

- **without compression**:
  - **~3 MB** of storage
  - **~22 seconds** for **transmission**, on 1Mbps
- **compressed** at 10:1 compression ratio
  - storage = **~300 KB**
  - **transmission** time drops to **~2.3 seconds**
- **Compression time**: **ten 1MB images** compressed and saved in ***less time*** than transferring **1 uncompressed file over the network!**

Content Dependence

Compressed file sizes vary depending on detail, colours, and information in the image, i.e., how much redundancy and repetition is present.

Example

1.3 MB for 80% Quality JPEG for a 5 mega-pixel image

- if image has large uniform areas (e.g., blue or grey skies), maybe 0.8 MB at 80% JPEG quality
- if image has lots of fine detail (e.g., floral foreground, trees etc.), maybe 1.7 MB at 80% quality

Why? encoding techniques are content-dependent

JPEG is Lossy

In terms of image quality, JPEG can be bad …

JPEG is Lossy

In terms of image quality, JPEG can be bad …

https://youtu.be/hWs0k__OX_w

Beware: Every time you save a JPEG image (after making a change…) data is re-compressed (a new approximation is used) and quality is reduced.

GIF - Graphic Interchange Format

- Limited to 256 colours only (8-bit)
  - uses lookup table (false colour mapping) to get 256 (only) 24-bit colour output

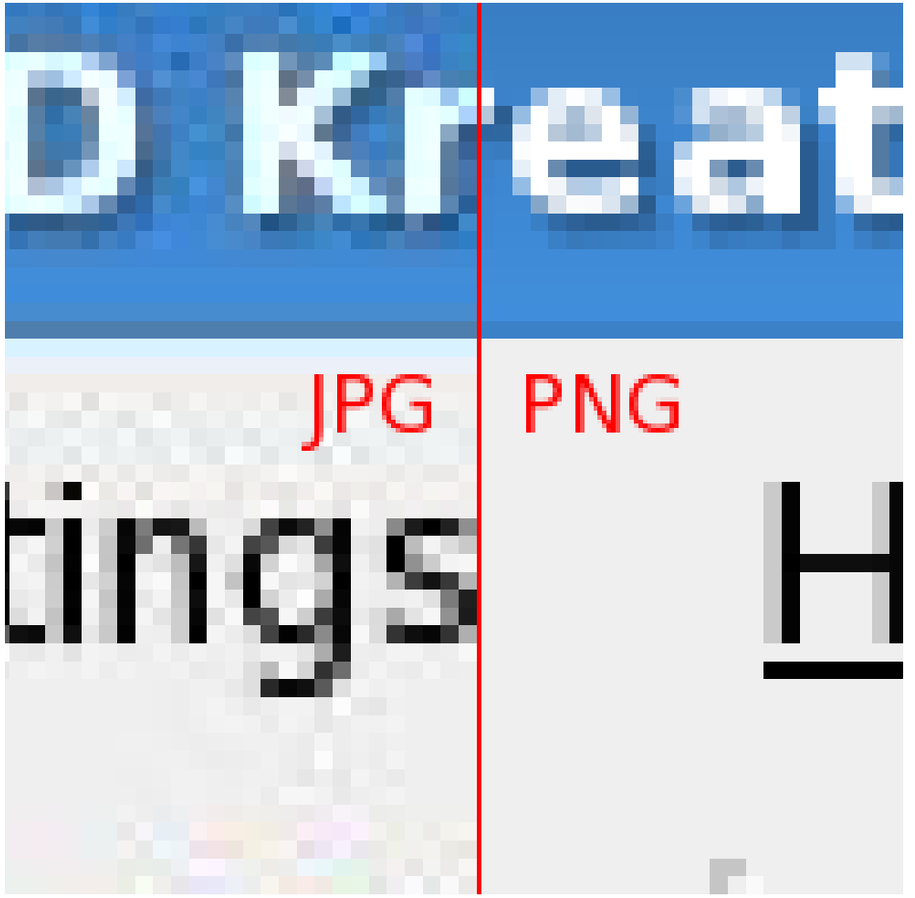
Lossless compression: patented LZW algorithm (general data compression, not image specific)

PNG - Portable Network Graphic

- Lossless compression
  - No changes to the data
- Image types:
  - Colour, grayscale and palette-based ("8-bit").
  - Designed as GIF replacement (JPEG too!)
- LZ77 algorithm (general compression)
  - **Huffman encoding**

Disadvantages:

- Larger file sizes.
- Slower read/write compression times.



Quick Demonstration

- Compression Artefacts:

https://github.com/atapour/ip-python-opencv/blob/main/compression-artefacts.py

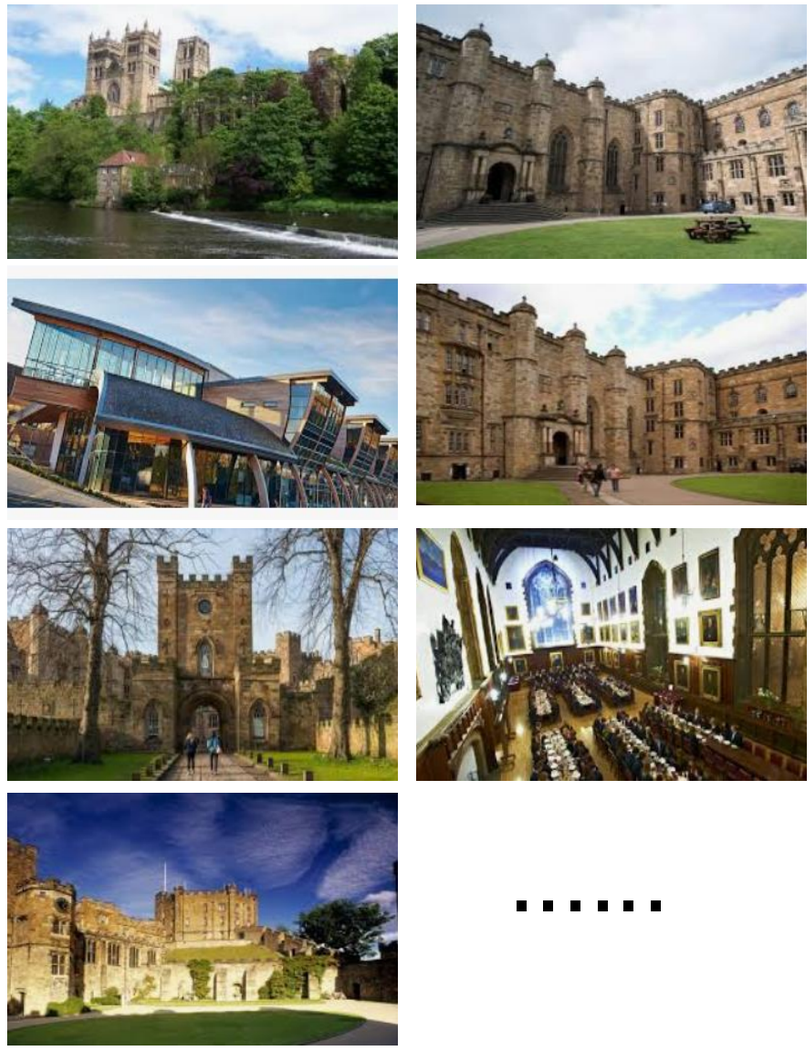
Summary

Image compression:

- Sources of Redundancy
- Lossy vs. Lossless

Techniques:

- DCT
- JPEG
- GIF
- PNG



.....


That's all Folks!